WEEKLY PROGRESS REPORT | WEEK 1

AI-Based Knowledge Retrieval Platform with Query Resolution System

Research findings and initial design for a text and voice question-answering tool built on RAG and cooperating agents.

INTERN
Manya N V

MENTOR
Vivek Gautam

PERIOD
Week 1

SUBMITTED
19 Oct 2026

01 Overview

People often need one specific answer buried inside a handbook, policy or manual, and scrolling or keyword search wastes time. This project builds a platform, not tied to any single subject area, where a user uploads documents and asks questions by typing or speaking.

Answers will come from Retrieval-Augmented Generation (RAG) combined with a few specialised agents. Every answer should display the passages it was based on and a relevance indicator, so users can verify it themselves.

This week at a glance

Activity	Status	Next step
Understanding project requirements	Done	Re-check against the project brief
Studying RAG and multi-agent ideas	Done	Keep notes handy for reference
Choosing a tool stack	Done	Confirm versions and compatibility
Drafting architecture and data model	Drafted	Refine while building
Document ingestion	Planned	Build and test in Week 2
Retrieval testing	Not started	Starts after indexing works

Besides the project research, I finished practice notebooks covering Python, object-oriented programming and SQL interview questions.

02 What I Studied

How RAG works

A language model alone answers only from its training. RAG adds a search step first: the most relevant passages from the user's own documents are found and given to the model as context, so the answer stays tied to real sources. There are two stages:

- **Indexing (done ahead of time):** load files, extract and clean text, split it into chunks, convert chunks to embeddings and store them with metadata.

- **Querying (done per question):** embed the question, fetch and rank the closest chunks, then pass them to the model.

Voice input and output

Browsers expose SpeechRecognition for speech-to-text and SpeechSynthesis for text-to-speech through the Web Speech API. Support varies by browser and needs microphone permission, so typing will always remain available.

Patterns for multiple agents

- **Sequential pipeline:** fixed order of steps, easiest to debug.
- **Router:** the kind of question picks the path.
- **Clarification:** unclear questions get a follow-up before searching.
- **Context:** recent turns help interpret follow-up questions.

Tools I shortlisted

Area	Choice	Reason
Chunking	Recursive character splitter	Splits at paragraph and sentence edges with some overlap
Embeddings	all-MiniLM-L6-v2	Compact model that can run locally
Vector store	ChromaDB	Vector search plus metadata storage
Retrieval	Dense top-k	Simple baseline before trying re-ranking
API	FastAPI	Good fit for upload and query endpoints
Voice	Web Speech API	Built into supporting browsers

These picks are a starting point and will be confirmed once the environment is set up.

03 Proposed Design

Layers

Layer	What it does	Components
Presentation	Chat window, upload, voice buttons, source display	HTML/CSS/JS or React
API	Takes uploads and questions, manages sessions	Python, FastAPI
Orchestration	Coordinates the agents and conversation context	Python orchestrator
Knowledge and data	Parsing, chunking, embeddings, vectors, logs	sentence-transformers, ChromaDB, SQLite

How a question travels through the system

1. User uploads a document.
2. The file is validated and its text and metadata extracted.
3. Text is cleaned, chunked, embedded and indexed.
4. User asks a question by text or voice.

5. The system interprets it and retrieves matching chunks.
6. An answer is generated from those chunks, with references.
7. The answer and sources are shown; weak matches are handled with care.

Agents

Agent	Job	Produces
Conversation Memory	Makes follow-up questions self-contained using earlier turns	Rewritten query
Query Understanding	Finds intent; labels it factual, procedural, comparative or ambiguous	Intent and route
Retrieval	Searches the vector store and selects relevant chunks	Ranked chunks
Clarification	Asks a short question when the query is unclear	Clarifying question
Response Generation	Writes the answer from retrieved text with citations	Answer and sources

Factual questions get a direct answer, procedural ones get ordered steps, comparative ones collect evidence for each side, and vague ones such as "What is the policy?" trigger a clarifying question. The retrieval score will act only as a relevance hint, not as a probability of correctness, and cut-offs will be set during testing.

Data model

Entity	Fields	Why
Document	doc_id, filename, file_type, upload_time	Track uploads
Chunk	chunk_id, doc_id, text, page_or_row, chunk_index	Text with its origin
Embedding	chunk_id, vector	Vector for each chunk
Session	session_id, created_at	Group a conversation
Message	msg_id, session_id, role, text, timestamp	Store turns
Retrieval Log	query, chunk_ids, scores, answered	Debugging and evaluation

Ingestion pipeline

Supported files: PDF (page numbers kept), DOCX, TXT, and CSV where each question-answer pair is one record.

Step	Action	Outcome
Validate	Check type, size and content	Accept or clear error
Parse	Extract text and read CSV records	Text with metadata
Clean	Normalise whitespace, fix artefacts	Cleaner text
Chunk	Split while keeping context and position	Linked chunks
Embed and index	Create vectors and store with metadata	Searchable knowledge base